X Article を読み解く / ジュニアエンジニア向け

AIが書くコードを、どうレビューするか

Anthropic「Claude Code」チームが実践する、6層のレビューと「終わらないレビュー」との戦い方

出典：Addy Osmani「How the Claude Code team does code review」（2026-10-26）

きょうの話は、３行でいうとこれ

01

人間が 1 行ずつ読むレビューは、もう回らない

Claude Code のコードの約 9 割は AI が書く。読む量が桁違いに増えた。

02

代わりに「6 つの層」で捕まえ、人間は危険な所だけ見る

1 つの関門ではなく、何段ものフィルターを通す。すり抜けたら戻す。

03

いちばんの敵は「AI 同士のレビューが終わらないこと」

実際に一晩で 50 往復した PR があった。上限と収束の仕掛けが要る。

何が起きているのか：数字で見る

6 月 → 9 月のあいだに、レビューの前提が壊れるほどの量になった

90%

Claude Code のコードのうち Claude が書く割合

2 倍超

マージされた PR の数（6 月 → 9 月）

40% 減

戻された PR の割合（revert）

30,000

社内モノレポで 1 日にマージされる変更数

「読む量」を減らしたのではない。読む場所を、危険な所に集めた。

※ この資料は記事の主張を紹介するもので、私たちのチームの決定ではない。

まず、問いを変える

これまでの問い

「誰かが、このコードを読んだか？」

↓

これからの問い

「この変更で、何が壊れうるか。
**　それを捕まえるのに、一番いいチェックは何か」**

「読んだか」は、測りたいことの代用にすぎない。
レビューは「最後の関門」から「書きながら回すスペルチェック」へ。

レビューには、目的が２つある

目的が２つあると分かると、何を機械に任せていいか判断できる

目的 1

バグを、出荷する前に捕まえる

記事の主役はこちら。
自動化で大半をカバーできる領域。

目的 2

コードベースの共通理解をつくる

こちらは後半で扱う。
「誰も読まなくなったら、どう学ぶのか」。

どちらの目的も「人がコードを読む」ことで達成されてきた。
だから「読む」を減らすとき、何が失われるかを意識して設計する必要がある。

1 つの関門ではなく、 6 つの層＋戻す仕組み

それぞれの層は別のバグを捕まえる。全部すり抜けたら、最後に戻す。

1. **PR 前の検証**　テスト・ビルド・スクショ・差分を、新しい目で読む
2. **AI レビュー**　全 PR に自動で。誤検知は複数エージェントの投票で落とす
3. **セキュリティ**　未解決の指摘があると、マージできない
4. **自動チェック**　lint ・型・全 OS のテスト・ビルドが通らないと進めない
5. **人間の承認**　リスクの高さで、要る承認の数と強さが変わる
6. **出し方**　マージキュー・フラグ・社内先行・段階リリース
7. **戻す**　事故に気づいたら、 1 手で戻せるようにしておく

※ 記事では 1〜6 を「層」、 7 を「最後の砦」として分けて書いている。

層 1 ： PR を作る「前」に、ほとんど終わらせる

ここが一番効く。レビューは PR を出してから始まるもの、ではない。

走らせて、結果を見せる

「終わりました」ではなく「これが結果です」。テスト・ビルド・
設計とのスクショ比較を、実際に動かして出す。

差分を、新しい目で読ませる

文脈を持たないサブエージェントに diff を読ませる。
書いた本人の思い込みが入らない。

社内スキルを回す

バグ探索・ diff の縮小・ E2E 検証・デザイン規約との照合を、
PR を出す前に実行する。

同じ指摘が続くなら、源を直す

レビュアーではなく、スキル・ lint ルール・ CLAUDE.md を直す。
これが「指摘が減っていく」仕組み。

層 2 ：すべての PR に、 AI レビューが入る

「全コードベース」と「 PR の意図」の両方に照らして diff を見る

数字で見る効果

16% → 54%

中身のあるレビューコメントが付く PR の割合。
1,000 行超の PR は大半で検出、小さい PR はクリーン。

誤検知を減らす仕組み

- 複数のエージェントが、並列でバグを探す
- 別の投票で「本物 / 誤検知」を判定する
- スタイル・診断・掃除だけの指摘は捨てる

指摘には必ずラベルが付き、重要度の順に並ぶ

Important
本当に直すべきもの

Nit
細かい指摘。好みの範囲

Pre-existing
元からあるバグ。著者の責任にしない

すべての指摘に向き合う：直すか、理由を書いてスレッドを閉じる。

放置はできない。だからこそ「指摘の質」がそのまま開発速度になる。

層３・層４：機械が「通さない」と言える関門

層３

セキュリティレビュー

- 未解決の指摘が１つでもあると、マージできない
- 直すか、短い理由を書いて閉じる（誤検知への対応）
- あとのコミットで直れば、自動でクローズされる

層４

自動チェック

- lint ・型チェック・全プラットフォームのテスト・ビルド
- CLI の PR だと、見る前に数十のチェックが走る
- PR テンプレに「どうテストしたか」「リスク」「戻し方」

関門が「ブロックする」ことに意味がある

止めるのが速いほど、直すコストは小さい。だから最初の層ほど大事。

層 5 〜 7 ：人間・出し方・事故のあと

残った仕事は「危険な変更を、正しい強さで見る」こと

層 5

人間の承認

段は「変更内容」で決まる（人ではない）。分類器が危険度を判定し、別の分類器が「もう 1 人の承認が要るか」を決める。

信頼リストの人だけ緩和。エージェントの承認は人間の承認として数えない。承認は 1 コミット限り。

層 6

出し方

マージキューで main に当ててから通す。フィーチャーフラグの裏で社内に先に出し、 latest → stable と段階的に広げる。

層 7

事故のあと

フラグなら配備なしで止められる。週次のオンコール。事故レビューで「どの層が捕まえるべきだったか」を振り返り、次の層を強くする。

いちばん怖いのは、レビューが終わらないこと

一晩で往復した回数

50 回以上

繰り返し指摘された行のうち、最初からそこにあったコード

約 59%

＝ 新しいバグではなく、
古いバグを毎回 1 個ずつ掘っていた

何が起きるか

AI の著者と AI のレビュアーは、永遠に往復できてしまう。実際に CLI の PR が一晩で 50 回以上サイクルし、手で止めた。

なぜ悪化したか

「回せば別のバグも見つかる」という理由で、既定で何度もレビューを回す習慣になっていた。

なぜ効かないか

git blame してみると、指摘の約 6 割は最初からあるコード。新しいバグはほとんど見つかっていなかった。

人の指摘との違い

人が付けた Nit は 1 秒で流せる。エージェントが付けた Nit は 1 コミットと 1 往復を生む。コストが全然ちがう。

だから「収束する」仕組みを入れた

ゴールは「全部見つける」ではなく「有限回で、正しい所に着く」

1. **サイクル数に上限を置く**
   3 回まで回しても終わらなければ、方針そのものを見直させる
2. **2 回目以降は、新しい差分だけ見る**
   全 diff を毎回読み直さない
3. **バグの「種類」をまとめて伝える**
   1 行ずつ直させず、同じ種類を一度に直させる
4. **既に出た指摘は、再報告しない**
   同じことを何度も言わない
5. **夜間に、現行コードをまるごと走らせる**
   歴史的なバグを、増える前に見つける

評価の軸も変えた：「何件見つけたか」ではなく「何回で通ったか」。

それでも、人間にしかできないこと

シニアがよくあるミスを直すのは、自動化するより高い

そもそも、作るべきか	構想・趣味・目標を決める。外のチームからの変更は今も人が見る
危険なコードの監査	課金・自動更新・ CI ワークフロー
アーキテクチャ	他の人が真似する決めごとになるから
証拠の確認	テスト出力・実行したコマンド・スクショ・レビューの指摘
責任を引き受ける	誰が打ったコードでも、出した人に説明責任がある

「そのチェックが
見逃すものが
説明できないなら、
👍 を求めては
いけない。

それは本物の

レビューが必要だ」

自動レビューは「低リスクな変更を通す」ことはできる。
でも、人が要る場面で人の代わりにはなれない。

読まなくなったら、どうやって学ぶのか

レビューには「共通理解をつくる」という 2 つ目の目的があった

正式な仕組みは、作らなかった。
チームの誰も、 Claude Code 全体を頭に入れていない。入れる必要もないと考えている。

01	**知りたい箇所を、 Claude に解説させる**	数分で、常に最新の解説が出てくる
02	**読み物か、小さな図解を作らせる**	自分の好みのスタイルで教えてもらう
03	**これから変えるモジュールで学ぶ**	些細な diff を 1 行ずつ読むより、はるかに学びが多い

＋ オープンなチャンネルで作業を見せること自体が、学習の場になっている。

06　人間の役割

マージした人が、最後まで持つ

You land it, you own it —— 壊れたら、基本は自分で直す。

なぜ破綻しないのか

変更が小さい
レビューも、デバッグも、戻すのも簡単

何でも戻せる
フラグ・ revert ・段階リリースで数分

爆発半径で基準が決まる
高いコードほど、見る目と検証が増える

テレメトリと社内利用
回帰はすぐ表に出る

犯人探しをしない
直した人が持つ。事故レビューは「系」を見る

「 AI で作り始めるのは安い。
でも、始めない判断も同じくらい大事」

作るのが簡単になったぶん、方向転換は早めに。

次の境界

低リスクな変更は、自動チェックが通れば自動マージしてよい——オーナーがそう決め始めている。

07　持ち帰り

明日からできること

特別な道具は要らない。ほとんどは、今のチームで始められる。

PR を出す前

- チェック項目を CLAUDE.md に短く列挙する
- フックで formatter と linter を自動で回す
- 新しい文脈のサブエージェントに差分を読ませる
- 繰り返す指摘は REVIEW.md に貯める

PR ごと

- CI を必須にする
- テンプレに Testing / Risk / 戻し方
- まず 1 つのリポジトリで AI レビュアーを試す
- セキュリティレビューとスレッド解決を必須に

承認

- 段は「変更内容」で決める（人ではなく）
- 緩和は信頼できる人だけ。厳しめから始める
- bot の承認は数えない
- 新しいコミットで承認を無効にする

マージしたあと

- マージキューを入れる
- フィーチャーフラグと段階リリース
- 1 手で戻せる仕組みを用意する
- 事故と revert を eval にする

まずは「1 つのリポジトリ・1 つの層」から。全部を一度に入れない。

まとめ

持ち帰るなら、この３つ

01

レビューは「関門」ではなく「層」

1 か所で捕まえようとせず、性質の違うチェックを重ねる。すり抜けたら戻す前提で作る。

02

人間の時間は、両側でコストになる

レビューに使う時間と、壊れたコードを直す時間。片方だけ減らしても、節約にはならない。

03

自動化は「収束」まで設計する

見つける力を上げるほど、往復が増える。上限と打ち切りをセットで入れる。

出典： Addy Osmani 「How the Claude Code team does code review」（X Article, 2026-10-26）／ 原典 claude.dev blog
※ 数値・事例はすべて記事の記載によるもの。